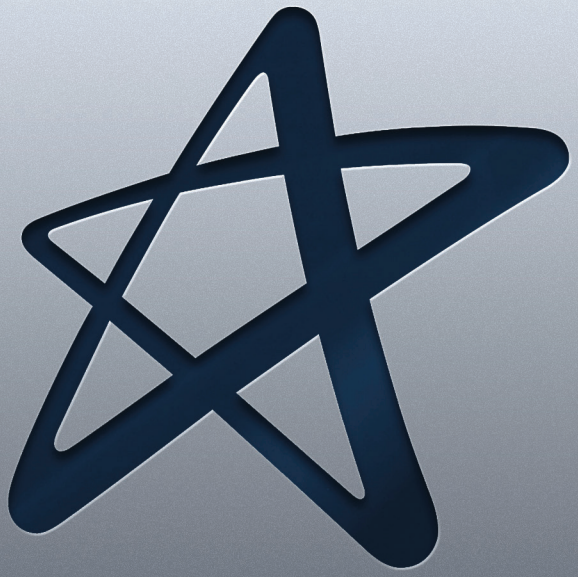
Linguagens Formais e Autômatos



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico

Linguagens Livres de Contexto

Responsável pelo Conteúdo:
Prof. Dr. Cleber Silva Ferreira da Luz

Revisão Textual:
Prof. Me. Luciano Vieira Francisco

UNIDADE Linguagens Livres de Contexto

- Introdução;
- Gramática Livre de Contexto.



OBJETIVO DE APRENDIZADO

- Conhecer os conceitos básicos sobre linguagens livres de contexto.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Nesta Unidade estudaremos um pouco mais sobre os mecanismos formais geradores de linguagens. Em particular, realizaremos um estudo sistemático sobre a Gramática Livre de Contexto (GLC), tratando-se de um formalismo gerador de linguagem. É uma gramática mais flexível em suas produções do que a gramática regular, porém, ainda com certas restrições.

Inicialmente, a GLC foi criada objetivando permitir o formalismo sintático das linguagens naturais – inglês, espanhol, português etc. –, no entanto, foi percebido que as linguagens naturais são mais complexas do que as concebidas a partir da gramática livre de contexto, diminuindo, assim, o interesse pelas gramáticas desses tipos (RAMOS; VEJA; JOSE, 2009).

Em contrapartida, a comunidade científica computacional se interessou pela GLC. Tal interesse surgiu quando foi observado que a gramática livre de contexto pode ser aplicada na formalização de linguagens artificiais, assim como as linguagens de programação (RAMOS; VEJA; JOSE, 2009).

Ademais, a GLC é utilizada principalmente em:

- Linguagens de programação;
- Aplicações de inteligência artificial; e
- Editores de texto.

Nas linguagens de programação, a gramática livre de contexto possui um papel fundamental na etapa de **análise sintática**, podendo ser utilizada na construção de analisadores sintáticos. Em um analisador sintático para uma linguagem de programação, a GLC pode ser útil no **balanceamento de parênteses** e nas **expressões aritméticas**, por exemplo – na próxima seção estudaremos a GLC com mais detalhes.

Gramática Livre de Contexto

GLC pode ser considerada como um formalismo gerador, uma gramática com restrições na forma das regras de produções, de modo que tais restrições são definidas de maneira mais livre que nas gramáticas regulares (MENEZES, 2011).

As gramáticas livres de contexto têm como principal forma:

$$A \rightarrow \alpha$$

Caso ocorra uma derivação de A, a variável A que é o contexto não dependerá de qualquer símbolo que a **anteceda**, ou a **suceda**; ou seja, será **livre de contexto**.

Uma GLC pode ser **representada** por uma **quadrupla** $G\ (V, T, P, S)$, onde:

- *V* é o conjunto finito dos símbolos **não terminais**;
- *T* é o conjunto finito dos **símbolos terminais** que correspondem ao **alfabeto da linguagem** definida pela gramática;
- *P* é o conjunto das **regras de produção** da gramática;
- *S* é a **raiz da gramática – variável inicial.**

Assim como uma gramática regular, mas com a restrição de qualquer regra de produção de *P*, temos:

$$A \rightarrow \alpha$$

Onde *A* é uma variável de *V* e α, uma palavra de (V U T)*.

É importante observar que uma GLC é uma gramática na qual o lado esquerdo das produções contém apenas uma variável (RAMOS; VEJA; JOSE, 2009). Para facilitar o entendimento, observemos o seguinte caso:

Exemplo 1

Considere a seguinte gramática livre de contexto:

$$G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$$

- **Onde:**
  - $P = \{E \rightarrow T + E$, 1
  - $E \rightarrow T$, 2
  - $T \rightarrow F * T$, 3
  - $T \rightarrow F$, 4
  - $F \rightarrow (E)$, 5
  - $F \rightarrow a\ \}$ 6

Consideremos a palavra *a * (a + a)*, que pode ser obtida a partir desta sequência de derivações:

$E \rightarrow T \rightarrow F * T \rightarrow a * T \rightarrow a * F \rightarrow a * (E) \rightarrow a * (T + E) \rightarrow a * (F + E) \rightarrow a * (a + E) \rightarrow a * (a + T) \rightarrow a * (a + F) \rightarrow a * (a + a)$

Sequência que corresponde às aplicações das regras de produções: 2, 3, 6, 4, 5, 1, 4, 6, 2, 4, 6, respectivamente.

A linguagem gerada por esta gramática – gramática exemplo – compreende as palavras que representam expressões aritméticas corretamente formadas sobre o operando *a* com operadores + e *. Tal gramática permite, também, criar **subexpressões** delimitadas por meio de parênteses. Ademais, essas subexpressões podem ser geradas com base nas mesmas regras utilizadas para construir a expressão inicial (RAMOS; VEJA; JOSE, 2009).

Notação BNF

Uma maneira usual de representar uma gramática livre de contexto é o uso da *Backus Naur Form* (BNF), de modo que:

- As **variáveis** são palavras delimitadas pelos símbolos < *T* >;
- As palavras **não delimitadas são terminais**;
- A regra de produção $A \rightarrow a$ é substituída por $A ::= a$;
- | é utilizado para separar os símbolos.

Para facilitar o seu entendimento, observemos os seguintes exemplos:

Exemplo 2

Dadas as seguintes regras de produção:

$F \rightarrow a$

$F \rightarrow b$

Temos as seguintes BNF:

<F> ::= a

<F> ::= b

Ou ainda:

<F> ::= a | b

Agora, consideraremos um caso dentro de um contexto claro e objetivo:

Exemplo 3

Suponha ser necessário definir uma BNF capaz de gerar qualquer identificador válido na linguagem de programação *Java* (MENEZES, 2011); assim:

- Toda letra é um identificador;
- Se *S* é um identificador, então a concatenação à direita de *S* com qualquer letra ou dígito também é um identificador.

Assim, temos uma BNF como esta:

<identificador> ::=<letra> | <identificador><letra> | <identificador><dígito>

<letra> ::= a | b | z

<dígito> ::= 0 | 1 | ... | 9

Árvore de Derivação

Algumas vezes pode ser útil representar a **estrutura de sentenças**, ou formas sentenciais de uma linguagem livre de contexto. Habitualmente, tal representação é realizada por meio de uma **árvore de derivação**, chamada também de **árvore sintática.**

Uma árvore de derivação é útil para representar as estruturas sentenciais, uma vez que:

- Viabiliza meios para melhor visualização da estrutura das sentenças das linguagens, facilitando a análise das mesmas;
- Ajuda na demonstração formal de teoremas, na interpretação de resultados e na assimilação de diversos conceitos;
- Simplifica a representação interna nos compiladores e interpretadores.

Em suma, uma árvore de derivação impõe **estruturas hierárquicas às sentenças das linguagens geradas** (RAMOS; VEJA; JOSE, 2009). Uma definição formal de árvore de derivação pode ser dada como um sistema de representação de sequências de derivações em uma gramática livre de contexto *G (V, T, P, S)*, valendo que:

- A **raiz é o símbolo inicial** da gramática;
- Os **vértices interiores** – ou nós internos – são **variáveis**; se *A* é um vértice interior e $X_1, X_2 \ldots X_n$, são, então, "filhos" de *A*; logo:
  - » $A \rightarrow X_1, X_2 \ldots X_n$ é uma produção da gramática;
  - » Os vértices $X_1, X_2 \ldots X_n$ são ordenados da esquerda para a direita.
- Os **vértices folhas** – ou **nós folhas** – são símbolos terminais ou símbolos vazios, de modo que o vazio é o único "filho" de seu "pai" ($A \rightarrow \varepsilon$).

A Figura 1 ilustra os elementos de uma árvore de derivação:

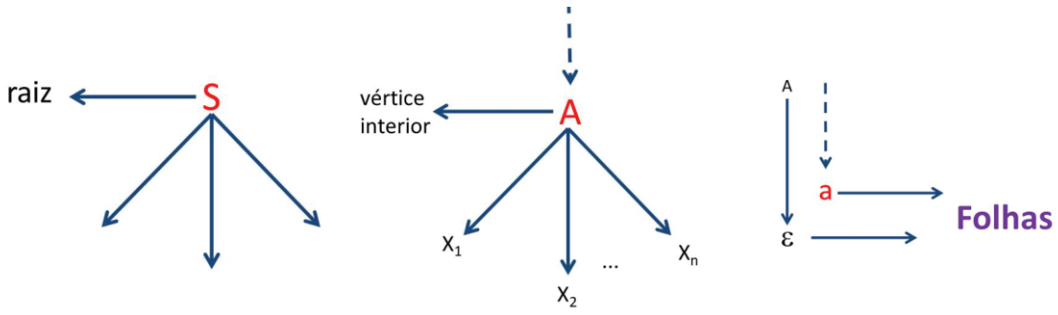


Figura 1 – Elementos da árvore de derivação
Fonte: Adaptado de Menezes, 2011

Exemplo 4

Dada a GLC apresentada anteriormente, criaremos a árvore de derivação para a palavra *a * (a + a)*. Assim, considere esta GLC:

$$G = (\{E, T, F\}, \{a, +, *, (,)\}, P, E)$$

- **Onde:**
  - » $P = \{E \rightarrow T + E$, 1
  - » $E \rightarrow T$, 2
  - » $T \rightarrow F * T$, 3
  - » $T \rightarrow F$, 4
  - » $F \rightarrow (E)$, 5
  - » $F \rightarrow a\ \}$ 6

Derivando a palavra *a * (a + a)* obtemos a seguinte árvore de derivação:

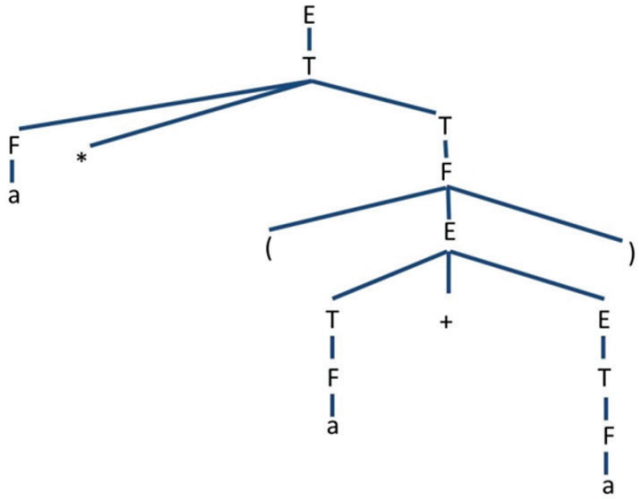


Figura 2 – Árvore de derivação para a palavra *a * (a + a)*
Fonte: Adaptado de Ramos, Veja e Jose, 2009

Árvore Abstrata

Trata-se de uma árvore gramatical na qual os operadores e as palavras-chave não aparecem como folhas, mas como nós interiores da árvore, figurando, assim, apenas o essencial.

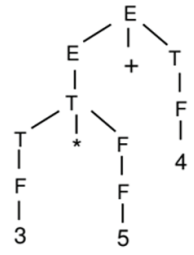


Figura 3 – Árvore de derivação
Fonte: Acervo do Conteudista

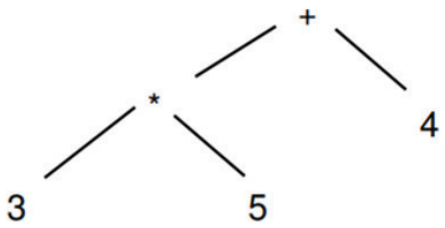


Figura 4 – Árvore Abstrata
Fonte: Acervo do Conteudista

Gramática Ambígua

A partir de uma **mesma palavra** algumas vezes pode-se obter **mais de uma árvore** devido à escolha do **lado da derivação – esquerdo ou direito**. Uma derivação mais à **esquerda** de uma palavra sempre obtém o símbolo não terminal mais à esquerda; já uma derivação mais à **direita** de uma palavra sempre obtém o símbolo não terminal mais à direita. Caso seja possível, em uma mesma palavra, obter duas ou mais árvores de derivação, tem-se caracterizada uma **gramática ambígua.**

Exemplo 5

Considere a seguinte GLC:

$$G = (\{E\}, \{a, +, -\}, P, E)$$

- **Onde:**
  - » $P = \{E \rightarrow E + E,$ 1
  - » $E \rightarrow E - E,$ 2
  - » $E \rightarrow a\}$ 3

Agora criaremos a árvore de derivação para a palavra $a + a - a$.

Se começarmos a derivar a partir da regra 1 teremos a árvore de derivação apresentada na Figura 5a; se escolhermos a regra 2 no começo da derivação teremos a árvore apresentada na Figura 5b. Logo, existe mais de uma árvore para essa GLC, tornando possível concluir se tratar de árvore ambígua.

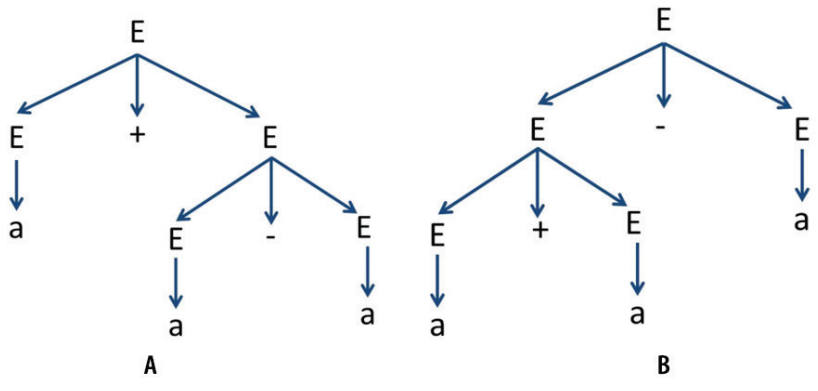


Figura 5 – Árvore de derivação para a palavra $a + (a - a)$
Fonte: Acervo do Conteudista

Simplificação de Gramática Livre do Contexto

Em alguns momentos, talvez seja conveniente simplificar a gramática livre de contexto. Em particular, é possível simplificar as regras de produção sem reduzir o poder de geração das GLC. Tais simplificações são utilizadas na construção e otimização de algoritmos (MENEZES, 2011). A seguir serão apresentadas as simplificações possíveis:

- **Eliminação de símbolos inacessíveis e inúteis:** exclusão de variáveis ou terminais não utilizados para gerar a palavra;
- **Eliminação de produções vazias:** exclusão de produções da forma $A \rightarrow \varepsilon$; se a palavra vazia pertencer à linguagem será incluída uma produção vazia específica para tal finalidade;
- **Produções que substituem variáveis:** exclusão de produções da forma $A \rightarrow B$, ou seja, que simplesmente substituam uma variável por outra.

Eliminação de Símbolos Inacessíveis

Esta simplificação consiste em excluir todas as produções que fazem referência a símbolos não utilizados na geração de palavras e excluir todos os símbolos inúteis. Para realizar tal simplificação, utilizaremos o algoritmo a seguir, o qual dividido em duas partes (MENEZES, 2011), de modo que qualquer:

- **Variável gera terminais:** o algoritmo reduz o conjunto de variáveis, analisando as produções da gramática a partir de terminais gerados. Inicialmente são consideradas todas as variáveis que geram terminais diretamente, por exemplo, $A \rightarrow a$;
- **Símbolo é atingível a partir do simbolo inicial:** após a execução da etapa anterior, o algoritmo analisa as produções da gramática a partir do símbolo inicial. Originalmente é considerado apenas o símbolo inicial para que, sucessivamente, as produções da gramática sejam aplicadas e os símbolos referenciados adicionados aos novos conjuntos.

Vejamos tal algoritmo com mais detalhes; para isso, considere uma GLC G = (*V, T, P, S*) a fim de aplicarmos a primeira etapa mencionada, ou seja:

- **Qualquer variável gera terminais.** A gramática resultante desta etapa é: $G_1 = (V_1, T, P_1, S)$, na qual $V_1, \subseteq V$. O conjunto P_1 contém os mesmos elementos que *P*, executando as produções cujas variáveis não pertencem a V_1. Para esta etapa o algoritmo é: $V_1 = \varnothing$; repita $V_1 = V_1 \cup \{A \mid A \rightarrow \alpha \in P \text{ e } \alpha \in (T \cup V_1)^*\}$ até que o cardinal de V_1 não aumente;
- **Qualquer símbolo é atingível a partir do símbolo inicial.** A gramática resultante desta etapa é: $G_2 = (V_2, T, P_2, S)$, na qual $V_2, \subseteq V_1$ e $V_2, \subseteq T$. O conjunto P_2 possui os mesmos elementos que P_1. O algoritmo desta etapa é: $T_2 = \varnothing$; $V_2 = \{S\}$. Repita $V_2 = V_2 \cup \{A \mid X \rightarrow \alpha\, A\beta \in P_1, X \in V_2\}$; $T_2 = T_2 \cup \{a \mid X \rightarrow \alpha\, a\beta \in P_1, X \in V_2\}$ até que os cardinais de V_2 e T_2 não aumentem.

Vamos a um exemplo para facilitar o seu entendimento:

Exemplo 6

Considere a seguinte GLC:

$$G = (\{S, A, B, C\}, \{a, b, c\}, P, S)$$

- **Onde:**
  - $P = \{S \rightarrow aAa \mid bBb$,
  - $A \rightarrow a \mid S$,
  - $C \rightarrow c\}$
- Qualquer variável gera terminais. Observe a Tabela 1, cuja coluna **Iteração** representa o número de ciclos do comando repita-se; e a coluna **Variáveis**, com o conjunto de variáveis, construído após as iterações. Vale ressaltar que o algoritmo para na terceira iteração, isto porque nenhuma variável foi adicionada ao conjunto (MENEZES, 2011);

  A produção $S \rightarrow bBb$ é excluída, pois B não pertence ao novo conjunto de variáveis. A gramática resultante da etapa 1 é: $G_1 = (\{A, C, S\}, \{a\}, \{S \rightarrow aAa, A \rightarrow a \mid S, C \rightarrow c\}, S)$.

Tabela 1 – Exclusão dos símbolos inúteis (etapa 1)

Iteração	Variáveis
Início	∅
1	{A, C}
2	{A, C, S}
3	{A, C, S}

Fonte: adaptada de Menezes, 2011

Qualquer símbolo é atingível a partir do símbolo inicial. Observaremos a Tabela 2, cuja produção $C \rightarrow c$ é excluída, pois C e c não pertencem aos novos conjuntos de variáveis e terminais, respectivamente. Nesta etapa, a gramática resultante é: $G_2 = (\{S, A\}, \{a\}, P, S)$; onde, $P = \{S \rightarrow aAa, A \rightarrow a \mid S\}$

Tabela 2 – Exclusão dos símbolos inúteis (etapa 2)

Iteração	Variáveis	Terminais
Início	{S}	∅
1	{S, A}	{a}
2	{S, A}	{a}

Fonte: adaptada de Menezes, 2011

Produções Vazias

A segunda fase da simplificação consiste em excluir as produções vazias, da forma A → ε (MENEZES, 2011). Para realizar tal simplificação, utilizaremos outro algoritmo, então dividido em três etapas:

- **Variáveis que constituem produções vazias:** inicialmente, considere todas as variáveis que geram diretamente a palavra vazia ($A \rightarrow \varepsilon$);
- **Exclusão de produções vazias:** são consideradas todas as produções não vazias;
- **Geração de palavras vazias, se necessário:** se a palavra vazia pertencer à linguagem, então será excluída uma produção para gerar a palavra vazia.

Vejamos o algoritmo desta fase:

- Consideremos uma GLC $G = (V, T, P, S)$. Como mencionado, o algoritmo é composto por três etapas:
  - **Eis o algoritmo da primeira etapa:** $V_\varepsilon = \{A \mid A \rightarrow \varepsilon\}$; repita $V_\varepsilon = V_\varepsilon \, U \, \{X \mid X \rightarrow X_1 ... X_n \in P$, tal que $X_1 ... X_n \in V_\varepsilon$ até que o cardinal de V_ε não aumente;
  - O algoritmo desta etapa é $P_1 = \{A \rightarrow \alpha \mid \alpha \neq \varepsilon\}$; repita para toda $A \rightarrow \alpha \in P_1$, $X \in V_\varepsilon$, tal que $\alpha = \alpha_1 X \alpha_2$, $\alpha_1 \alpha_2 \neq \varepsilon$; faça $P_1 = P_1 \, U \, \{A \rightarrow \alpha_1\alpha_2\}$ até que o cardinal de P_1 não aumente;
  - Geração de palavra vazia, se necessário: se a palavra vazia pertencer à linguagem, então a seguinte produção será incluída: $S \rightarrow \varepsilon$.

Produções que Substituem Variáveis

Esta simplificação consiste em substituir uma variável por outra.

Logo, produções do tipo $A \rightarrow B$ não adicionam informação alguma em termos de geração de palavras – a não ser o fato de que, neste caso, a variável A poderá ser substituída por $A \rightarrow \alpha$.

Formas Normais

As formas normais concedem rígidas restrições na forma de produção, sem reduzir o poder de geração das GLC. Habitualmente, são utilizadas no desenvolvimento de algoritmos (MENEZES, 2011). Existem duas formas normais, as de:

- **Chomsky**, na qual as produções são: $A \rightarrow BC$ ou $A \rightarrow a$;
- **Greibach**, com produções assim montadas: $A \rightarrow a\alpha$.

Existem ainda algumas etapas para converter uma GLC em cada uma das formas normais, de modo que veremos a forma normal de Chomsky com mais detalhes.

Forma Normal de Chomsky

Uma gramática livre de contexto G é dita estar na forma normal de Chomsky se todas as suas produções são assim estruturadas:

$$A \rightarrow BC \text{ ou } A \rightarrow a$$

Logo, uma palavra vazia não pertence à linguagem gerada por uma gramática da forma de Chomsky (MENEZES, 2011). As etapas para transformar uma GLC na forma normal de Chomsky são apresentadas a seguir:

1. **Simplificação da gramática:** esta fase exclui todas as produções vazias, da forma $A \rightarrow \varepsilon$ e $A \rightarrow B$ – caso o lado direito da produção tenha apenas um símbolo não terminal;

2. **Transformação do lado direito das produções de comprimento maior ou igual a dois:** garante que esta condição seja composta exclusivamente por variáveis. Nesta etapa, a exclusão de um terminal *a* pode ser realizada, substituindo-o por uma variável intermediária C_a e incluindo a produção $C_a \rightarrow a$;
3. **Transformação do lado direito das produções de comprimento maior ou igual a 3, em produções com exatamente duas variáveis:** garantindo esta condição, após a execução desta fase, o lado direito das produções da forma $A \rightarrow B_1 B_2 ... B_n (n \geq 2)$ é composto exclusivamente por variáveis. Assim, para terminar a transformação é suficiente garantir que o lado direito seja composto por exatamente duas variáveis, gerando-se $B_1 B_2 ... B_n$ em diversas etapas e utilizando duas variáveis intermediárias.

Forma Normal de Greibach

Uma GLC G é dita estar na forma de Greibach se todas as suas produções estiverem assim formadas:

$$A \rightarrow a\alpha$$

Onde *A* é uma variável de *V*, *a* é um terminal de *T* e α é uma palavra de V^*. Assim, a palavra vazia não pertence à linguagem gerada por uma gramática na forma normal de Greibach.

Para uma GLC G estar na forma normal de Greibach, torna-se necessário seguir estas etapas:

1. **Simplificação da gramática:** deve-se simplificar a gramática livre de contexto;
2. **Renomeação das variáveis em uma ordem crescente qualquer:** as variáveis da gramática são ordenadas de modo crescente, tais como A_1, $A_2 ... A_n$;
3. **Transformação de produções para a forma $A_r \rightarrow As\alpha$, na qual, $r \leq s$:** nesta fase as produções modificadas garantem que a primeira variável do lado direito seja maior ou igual à do lado esquerdo;
4. **Exclusão dos recursos das recursões da forma $A_r \rightarrow Ar\alpha$:** as recursões – à esquerda – podem existir originalmente na gramática, ou serem geradas pela etapa anterior. Assim, a eliminação da recursão à esquerda pode ser realizada, introduzindo-se variáveis auxiliares e incluindo recursão ($B_r \rightarrow \alpha B_r$);
5. **Um terminal no início do lado direito de cada produção:** após a execução da quarta etapa, todas as produções da forma $A_r \rightarrow As\alpha$ são tais que $r \leq s$. Logo, as produções da maior variável A_n só podem iniciar por um terminal do lado direito. Dessa forma, $A_{n-1} \rightarrow A_n\alpha$ é substituído A_n pelas suas correspondentes produções ($A_n \rightarrow a\beta\alpha$);
6. **Produções na forma $A \rightarrow a\alpha$ em que α é composto por variáveis.**

Recursão à Esquerda

Quando se cria um algoritmo para reconhecer uma linguagem, comumente no desenvolvimento de tais algoritmos é desejado que a gramática não seja recursiva à esquerda (MENEZES, 2011). Todavia, inicialmente é fundamental entender o que é uma recursão à esquerda, ocorrendo quando:

$$A \rightarrow {}^{+}A\alpha$$

Isto é, quando uma variável deriva de si, de modo direto ou indireto, como o símbolo mais à esquerda de uma subpalavra gerada.

Vale ressaltar que a transformação de uma gramática equivalente sem recursão à esquerda pode ser realizada executando-se as quatro primeiras etapas do algoritmo referente à forma normal de Greibach.

Autômato com Pilha

Assim como as linguagens regulares, as linguagens livres de contexto podem ser associadas a um formalismo do tipo autômato, sendo tal formalismo denominado **autômato com pilha** (MENEZES, 2011).

O autômato com pilha é análogo ao autômato finito, incluindo as formas de representação e a facilidade do não determinístico; todavia, é acrescentada uma pilha como memória auxiliar. Nesse formalismo, a pilha é independente da fita de entrada e não possui limite de tamanho, implicando na noção de "tão grande quanto se queira".

Estruturalmente, a sua principal característica é que o último símbolo gravado é o primeiro a ser lido. Na pilha, a sua base é fixa e define o seu início. O topo é uma variável e estabelece a posição do último símbolo gravado (Figura 6).

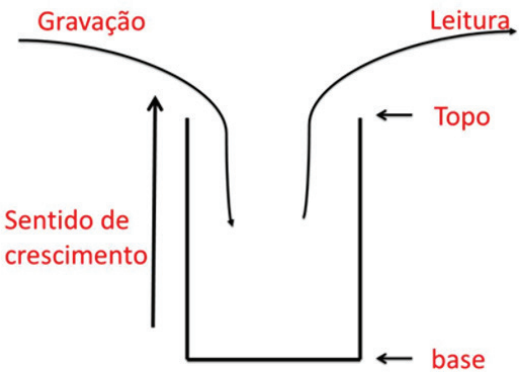


Figura 6 – Estrutura da pilha
Fonte: Adaptado de Menezes, 2011

Um autômato finito não determinístico é frequentemente utilizado em autômatos com pilha, pois a sua facilidade de reconhecimento de palavras facilita justamente o autômato com pilha. A próxima seção apresentará uma definição formal de autômato com pilha.

Definição de Autômato com Pilha

Formalmente, um autômato com pilha possui duas definições universalmente aceitas e que especificam o critério de parada do autômato, vejamos:

- O valor inicial da pilha é vazio e o autômato para, aceitando ao atingir um estado final;
- No início, a pilha contém um símbolo especial; não existem estados finais e o autômato para, aceitando quando a pilha estiver vazia.



Importante!

Essas duas definições são equivalentes.

Ademais, um autômato com pilha não determinístico, ou simplesmente um autômato com pilha é composto por:

- **Fita:** assim como um autômato qualquer;
- **Pilha:** uma memória auxiliar do tipo pilha que é utilizada para a leitura e gravação;
- **Unidade de controle:** que reflete o estado corrente da máquina; possui uma cabeça de fita e uma cabeça para a pilha;
- **Programa, função programa, ou função de transição:** que comanda a leitura da fita, leitura e gravação da pilha e define o estado da máquina.

Vale ressaltar que a pilha é dividida em células, armazenando, cada qual, um símbolo do alfabeto. A leitura ou gravação é sempre no topo. Não possui tamanho fixo ou máximo, sendo o seu tamanho corrente igual ao da palavra armazenada. O seu valor inicial é vazio.

Por sua vez, a unidade de controle possui um número finito de estados; possui uma cabeça de:

- **Fita:** análoga à cabeça da fita de um autômato qualquer;
- **Pilha:** unidade de leitura e gravação que se move para a esquerda – ou "para cima" – ao gravar, e para a direita – ou "para baixo" – ao ler um símbolo. Acessa um símbolo de cada vez, estando sempre no topo.

Ademais, o programa é uma função parcial tal que dependendo do estado corrente, os símbolos lidos da fita e pilha determinam o novo estado e a palavra a ser gravada – na pilha.

Formalmente, um autômato com pilha é uma 6-upla:

$$M = (\Sigma, Q, \delta, q_0, F, V)$$

- **Onde:**
  - Σ é o alfabeto de entrada;
  - Q é o conjunto finito de estados possíveis do autômato;
  - δ é uma função programa, chamada também de **função transição**;

» q_0 é um elemento distinguindo de Q, chamado de **estado inicial**;

» F é um subconjunto de Q, chamado, coletivamente, de **estados finais**;

» V é um alfabeto auxiliar, ou alfabeto da pilha.

A computação de um autômato com pilha, para uma palavra w qualquer, consiste na sucessiva aplicação da função programa para cada símbolo de w, até ocorrer uma condição de parada. Todavia, é possível que um autômato nunca alcance uma condição de parada. Caso isso ocorra, o processamento ficará em *loop* infinito.

Ao final do processamento pode ocorrer de:

- **Aceitar a entrada *w*:** pelo menos um dos caminhos alternativos atender a um estado final, de modo que o autômato parará e a palavra w será aceita;
- **Rejeitar a entrada *w*:** em que todos os caminhos alternativos rejeitarão a entrada, de modo que o autômato parará e a palavra será rejeitada;
- **Ficar em *loop* para a entrada *w*:** pelo menos um caminho alternativo estará em *loop* e os demais rejeitarão a entrada.

A Figura 7 ilustra o diagrama de um autômato com pilha:

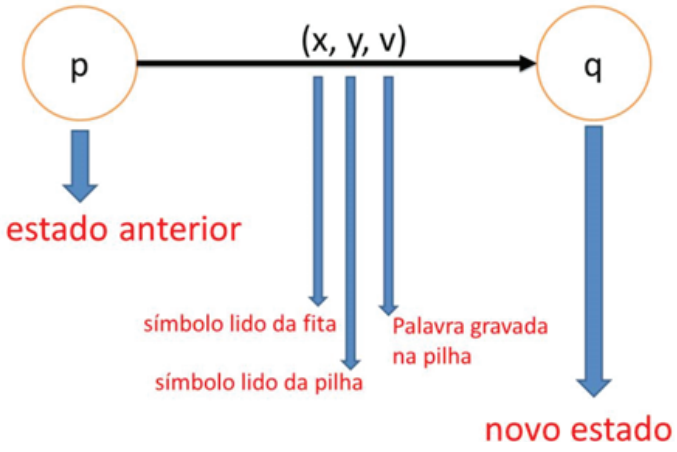


Figura 7 – Diagrama de um autômato com pilha
Fonte: Adaptado de Menezes, 2011

Autômato com Pilha e Linguagem Livre de Contexto

Dada uma GLC é possível construir um autômato com pilha equivalente. Assim, devemos observar que qualquer linguagem livre de contexto pode ser aceita por um autômato com pilha com somente um estado de controle lógico, significando que a facilidade de memorização de informações através de estados não aumenta o poder computacional dos autômatos com pilhas (MENEZES, 2011). Vejamos como tal construção é realizada:

Se *L* é uma linguagem livre de contexto, então existe *M*, autômato com pilha tal que ACEITA(M) = L.

Para isso, suponhamos uma palavra vazia que não pertence a *L* na perspectiva da construção de um autômato com pilha a partir da gramática transformada na forma normal de Greibach.

O autômato com pilha gerado simula a derivação mais à esquerda, em que:

- Lê o símbolo *a* da fita;
- Lê o símbolo *A* da pilha;
- Empilha a palavra α.

Tal simulação é realizada para cada produção, utilizando um único estado de controle. A construção do autômato com pilha *M* a partir da gramática G = (*V, T, P, S*) é como se segue – e ilustrado na Figura 8:

- Seja $G_{FNG} = (V_{FNG}, T_{FNG}, P_{FNG}, S)$, a transformação de *G* na forma normal de Greibach;
- Seja $M = (V_{FNG}, \{q_0, q_1, q_f\}, \delta, q_0, \{q_f\}, V_{FNG})$, em que:
  - » $\delta(q_0, \varepsilon, \varepsilon) = \{(q_1, S)\}$
  - » $\delta(q_a, a, A) = \{(q_1, \alpha) \mid A \rightarrow a\alpha \in P_{FNG}\}$
  - » $\delta(q_1, ?, ?) = \{(q_f, \varepsilon)\}$

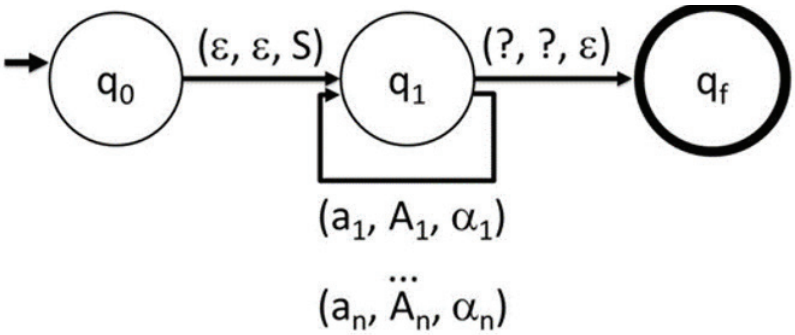


Figura 8 – Diagrama (AP): a partir de uma gramática na forma normal de Greibach
Fonte: Adaptado de Menezes, 2011

Propriedades das Linguagens Livres de Contexto

Tais propriedades permitem demonstrar se determinada linguagem é livre de contexto ou não (MENEZES, 2011). Assim como as linguagens regulares, as linguagens livres de contexto possuem um lema de bombeamento – estudado na Unidade anterior –, vejamos:

Se L é uma linguagem livre de contexto, então existe uma constante n tal que para qualquer palavra w de L onde $|w| \geq n$, w pode ser definida com $w = u\ x\ v\ y\ z$ na qual:

- $|x\ v\ y| \leq n$,
- $|x\ y| \geq 1$

Sendo que para todo $i \geq 0$, $u\ x^i\ v\ y^i\ z$ é palavra de L.

Em Síntese

A gramática livre de contexto é um formalismo gerador de linguagem. Trata-se de uma gramática mais flexível em suas produções do que a gramática regular, porém, ainda com certas restrições. Assim, uma GLC pode ser aplicada na formalização de linguagens artificiais, tais como as de programação.

Ademais, as gramáticas livres de contexto têm como principal forma a seguinte:

$$A \rightarrow \alpha$$

Caso ocorra uma derivação de A, a variável A que é o contexto não dependerá de qualquer símbolo antecessor ou sucessor, mantendo-se livre de contexto.

Uma maneira usual de representar uma gramática livre de contexto é o uso da forma *Backus Naur Form* (BNF), dado que algumas vezes pode ser útil representar a estrutura de sentenças ou formas sentenciais de uma linguagem livre de contexto. Habitualmente, essa representação é realizada por meio de uma **árvore de derivação, chamada também de árvore sintática**; já uma **árvore abstrata** é uma árvore gramatical, na qual os operadores e as palavras-chave não aparecem como folhas, mas como nós interiores da árvore – figurando apenas o essencial.

A partir de uma mesma palavra algumas vezes pode-se obter mais de uma árvore devido à escolha do lado da derivação – esquerdo ou direito. Caso seja possível, em uma mesma palavra, obter duas ou mais árvores de derivação, tem-se caracterizada uma **gramática ambígua.**

Em alguns momentos, talvez seja conveniente **simplificar a gramática livre de contexto**; em particular, é possível simplificar as regras de produções sem reduzir o poder de geração das GLC. As simplificações de uma GLC são utilizadas na construção e otimização de algoritmos.

Habitualmente, no desenvolvimento de algoritmos de reconhecedores de gramática são utilizadas as formas normais, dado que concedem rígidas restrições na forma de produção, sem reduzir o poder de geração das GLC.

Assim como as linguagens regulares, as linguagens livres de contexto podem ser associadas a um formalismo do tipo autômato, então denominado **autômato com pilha.**

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Introdução à teoria de autômatos, linguagens e computação

HOPCROFT, J. E.; ULLMAN, J. D.; MOTWANI, R. **Introdução à teoria de autômatos, linguagens e computação**. Rio de Janeiro: Campus, 2002.

Linguagens formais e autômatos

MENEZES, P. B. **Linguagens formais e autômatos**. 6. ed. São Paulo: Bookman, 2011.

Theory of computation

SIPSER, M. ***Theory of computation***. India: [s.n.], 2008.

Introdução à teoria da computação

SIPSER, M. **Introdução à teoria da computação**. 2. ed. São Paulo: Thompson Learning, 2007.

Referências

GERSTING, J. **Fundamentos matemáticos para a Ciência da Computação**. 5. ed. Rio de Janeiro: LTC, 2004.

HOPCROFT, J. E. ***Introduction to automata theory, languages and computation***. Massachusetts, USA: Addison-Wesley, 1979.

_____.; ULLMAN, J. D.; MOTWANI, R. **Introdução à teoria de autômatos, linguagens e computação**. Rio de Janeiro: Campus, 2002.

MENEZES, P. B. **Linguagens formais e autômatos**. 6. ed. São Paulo: Bookman, 2011.

PAPADIMITRIOU, C. H.; LEWIS, H. R. **Elementos da teoria da computação**. 2. ed. Porto Alegre, RS: Bookman, 2000.

RAMOS, M. V. M.; VEGA, I. S.; JOSE NETO, J. **Linguagens formais**: teoria, modelagem e implementação. Porto Alegre, RS: Bookman, 2009.

SIPSER, M. **Introdução à teoria da computação**. 2. ed. São Paulo: Thompson Learning, 2007.

VIEIRA, N. J. **Introdução aos fundamentos da computação**: linguagens e máquinas. São Paulo: Thompson Learning, 2006.


Cruzeiro do Sul
Educacional